Base de Datos Documentales

Práctica 6

Juan Sebastian Vasquez Cossio - 2240518

Resumen

En esta práctica conocerás qué es una base de datos documental y cómo crear una api RESTful utilizando el framework express.js y código en JavaScript para brindar un backend de conexión a la base de datos.

Duración
45 min

Autor
Juan Vicente Pradilla Cerón

Fecha de Creación
15/09/2022

Licencia

1. Bases de Datos Documentales

Duración: 10 min

Las **Bases de Datos Documentales** o **Bases de Datos Orientadas a Documentos** son una subcategoría de las **bases de datos no relacionales**; cuyo concepto central para el almacenamiento de datos es el **Documento**. Los Documentos siguen **formatos estandarizados**, como puede ser **XML**, **JSON** o **BSON**, pero la **estructura interna** de los mismos, no requiere ajustarse a un **esquema** rígido, dando así una gran flexibilidad al almacenamiento de datos. Como ejemplo tenemos los siguientes documentos, los mismos siguen un formato JSON pero la estructura interna es ligeramente diferente.

```
{
    "_id": 120000121200,
    "nombre": "Sensor de Temperatura - UAO - InfoLab",
    "mediciones": [{
                "_id": 230000232300,
                "momento": "2020-04-12T12:53:22.647Z",
                "valor": 33.2
            }
    ]
}
```

```
{
```

```
    "_id": 120000121201,
    "nombre": "Sensor de Humedad - UAO - InfoLab",
    "descripcion": "Mide la humedad del InfoLab en la sede Valle del Lili de la UAO"
}
```

Existe una gran variedad de **gestores** de Bases de Datos Orientadas a Documentos, siendo los más relevantes: DynamoDB, CouchDB, **MongoDB** y **Redis**; además, **PostgreSQL** soporta este tipo de almacenamiento de información. En general los diferentes gestores de Bases de Datos Documentales difieren en la **organización** de los Documentos optando por: **Colecciones**, Etiquetas, Metadatos o Jerarquías de directorios.

2. Implementando una API para una Base de Datos Documental

Duración: 35 min

Iniciar el gestor de base de datos

En esta práctica utilizaremos como gestor de base de datos **MongoDB** mediante su servicio de base de datos en la nube **MongoDB Atlas**.

1. Desde el navegador Web **accede** a la página Web de [MongoDB Atlas]
2. **Crea una cuenta**, dando clic al botón **Try Free** que se encuentra en la esquina superior derecha de la página (Fig. 1)

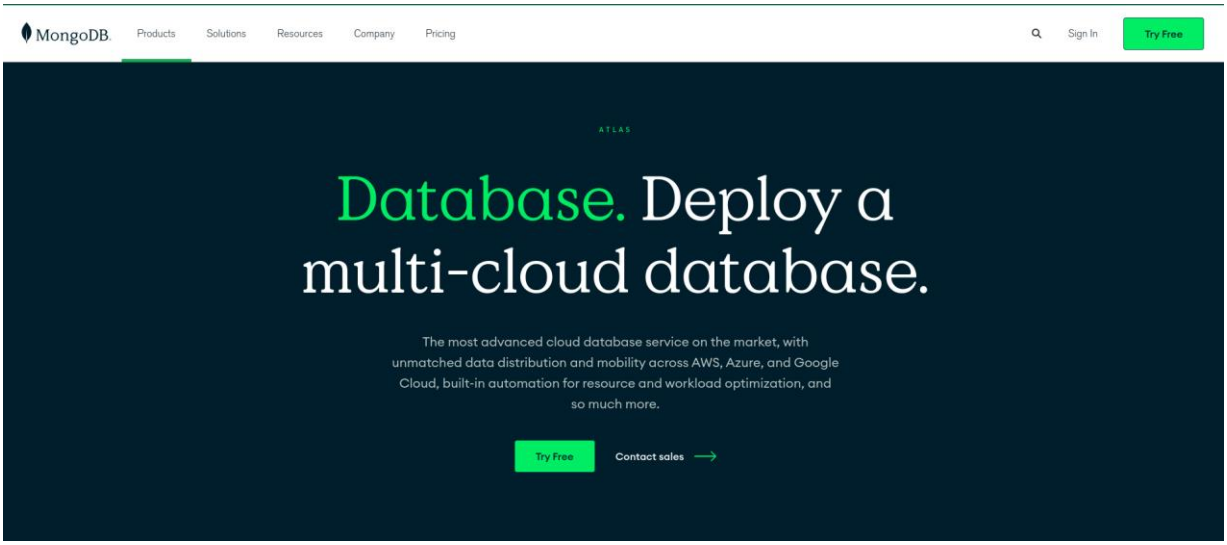


Figura 1. Crear una cuenta en MongoDB Atlas - Paso 1

3. A continuación, **llena el formulario** de creación de cuenta con los datos de tu cuenta institucional de la UAO. Selecciona que estás de acuerdo con los términos de servicio (recuerda leerlos) y presiona el botón **Create your Atlas account** (Fig. 2)

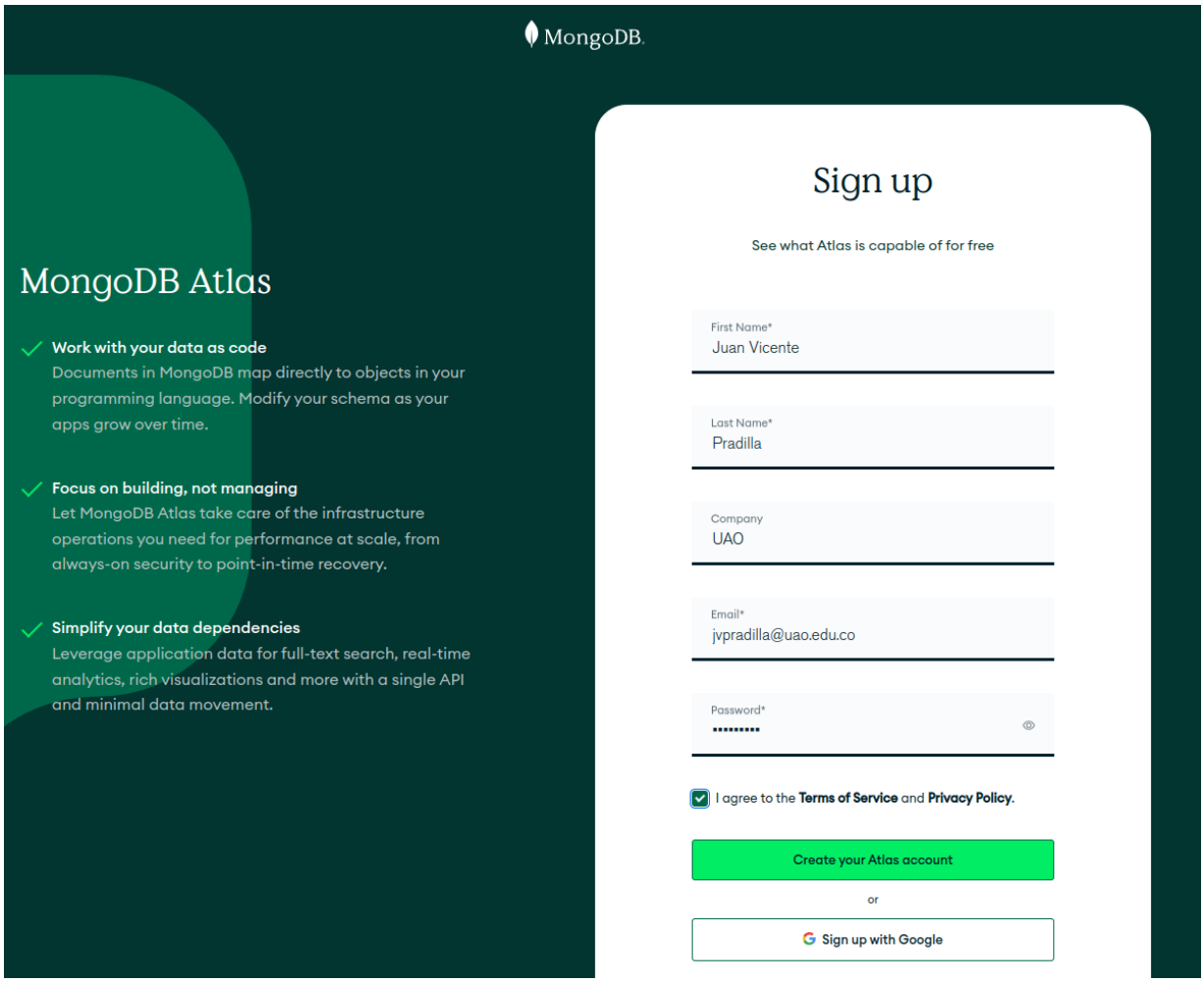


Figura 2. Crear una cuenta en MongoDB Atlas - Paso 2

4. Acto seguido, **llena el formulario** de creación del proyecto. Como organización puedes poner **UAO**, como nombre del proyecto **Practica 6** y como lenguaje de programación **JavaScript**. Luego presiona el botón de **Continue** (Fig. 3)

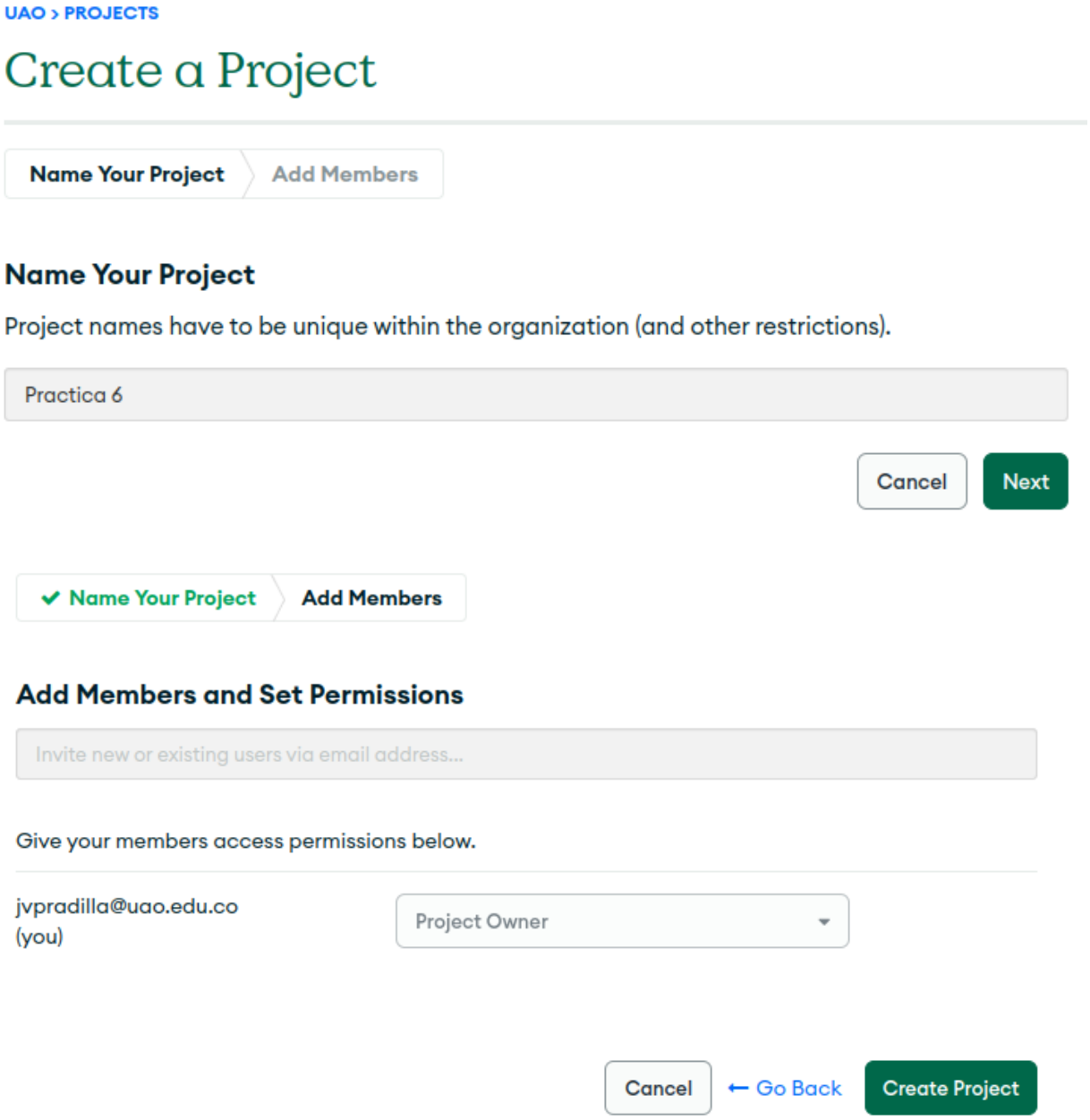


Figura 3. Crear un proyecto en MongoDB Atlas

5. Ahora, **selecciona el plan Shared Clusters** (gratuito) para crear un Cluster para el proyecto (Fig. 4)

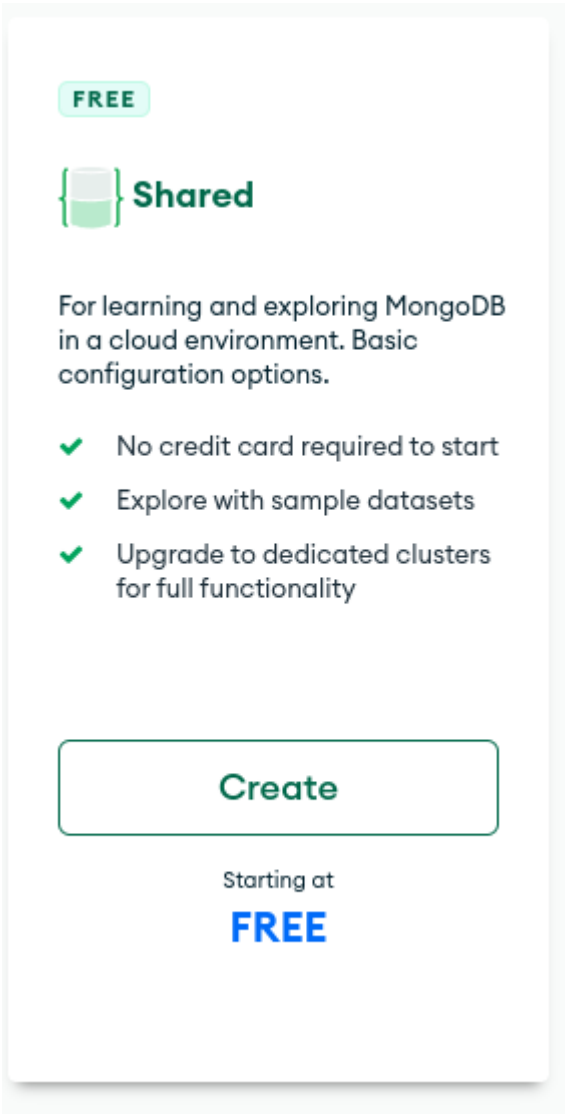


Figura 4. Crear un Cluster para el proyecto en MongoDB Atlas - Paso 1

6. Finalmente, en la **configuración del Cluster** deja los valores por defecto (Fig. 5)

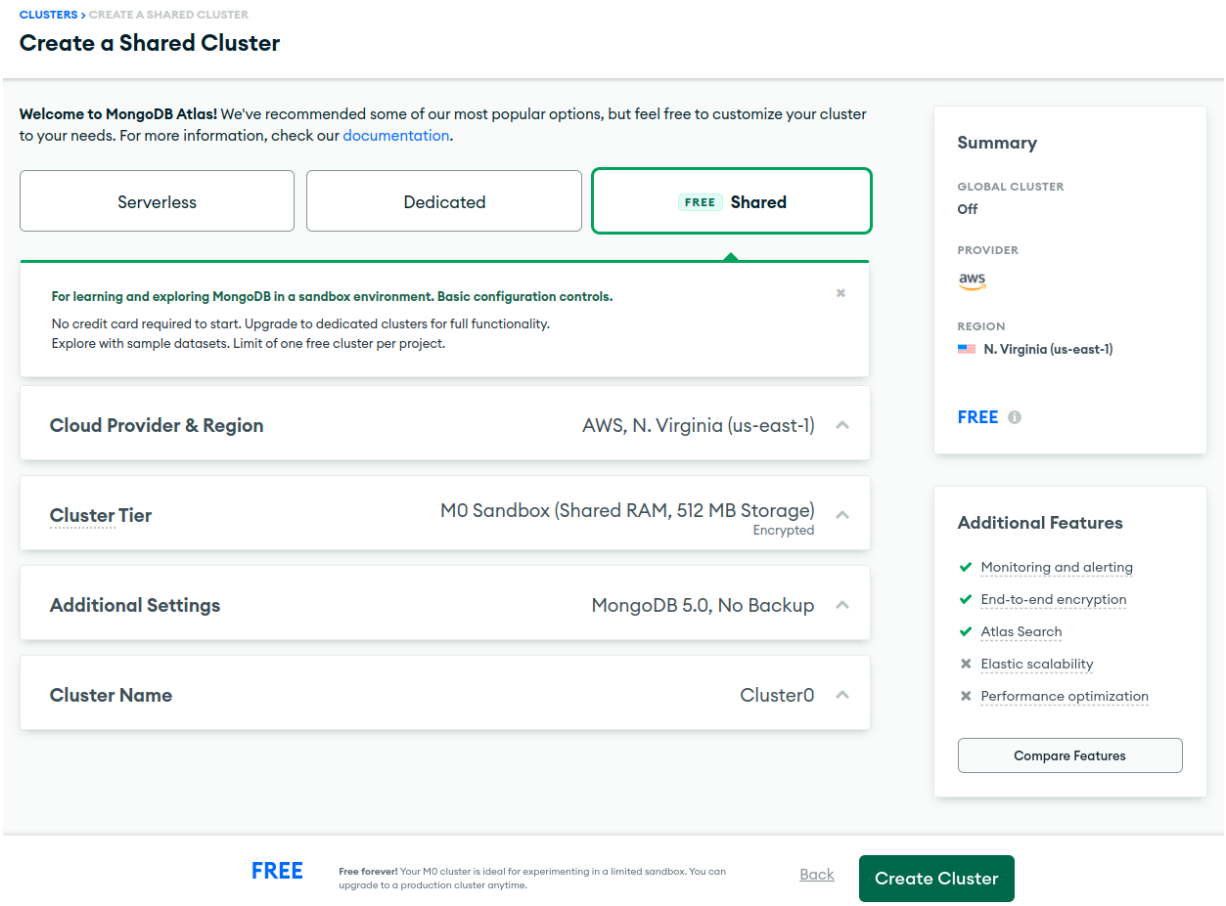


Figura 5. Crear un Cluster para el proyecto en MongoDB Atlas - Paso 2

7. Espera unos minutos a que el Cluster sea creado y configurado
8. Una vez que el Cluster se encuentre en funcionamiento presione el botón CONNECT, para configurar la conexión al Cluster (Fig. 6)

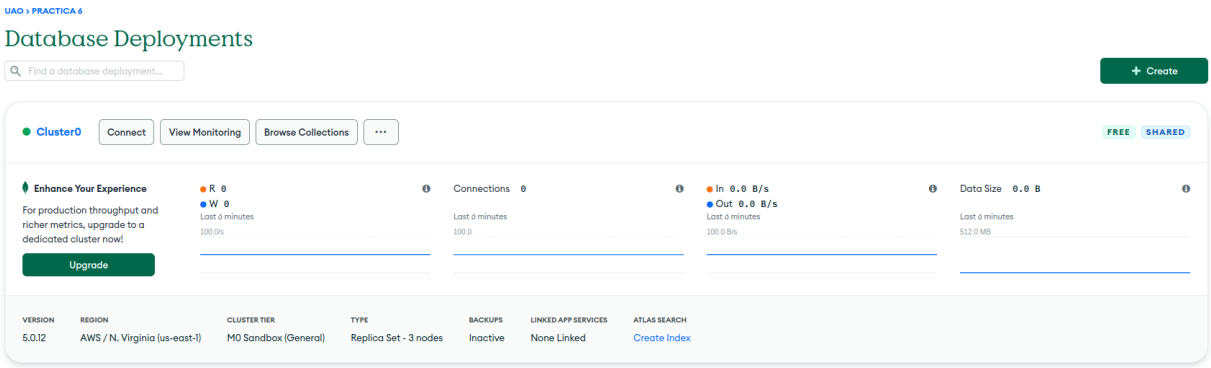


Figura 6. Cluster en funcionamiento en MongoDB Atlas

9. En la ventana que se abre en el punto 1 **selecciona Allow Access from Anywhere** (esta opción para el tutorial es válida pero en entornos de despliegue no se debe escoger) (Fig. 7)

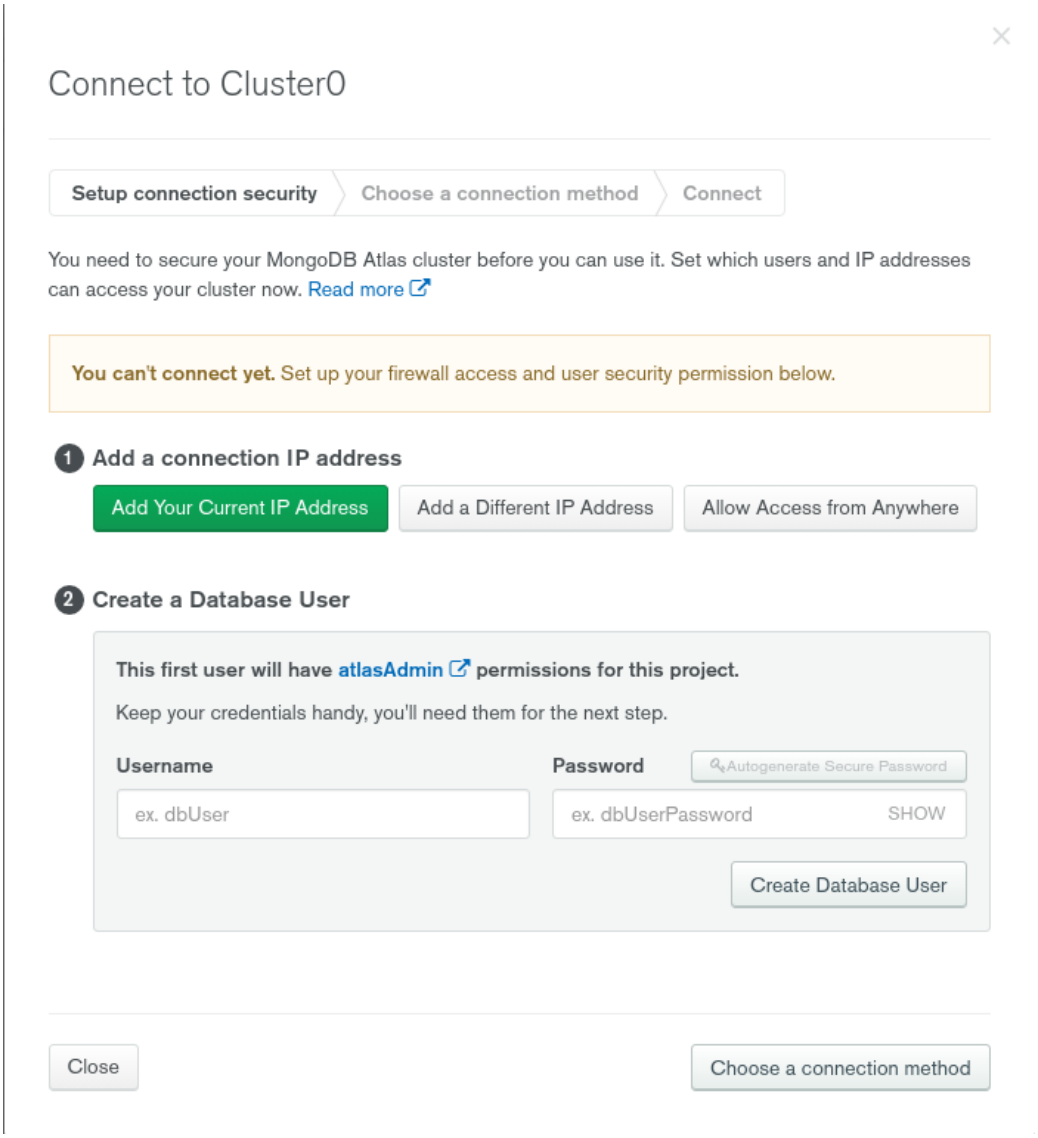


Figura 7. Configurar la conexión al Cluster en MongoDB Atlas - Paso 1

10. Selecciona **Add IP Address** en el panel que se desplegó (Fig. 8)

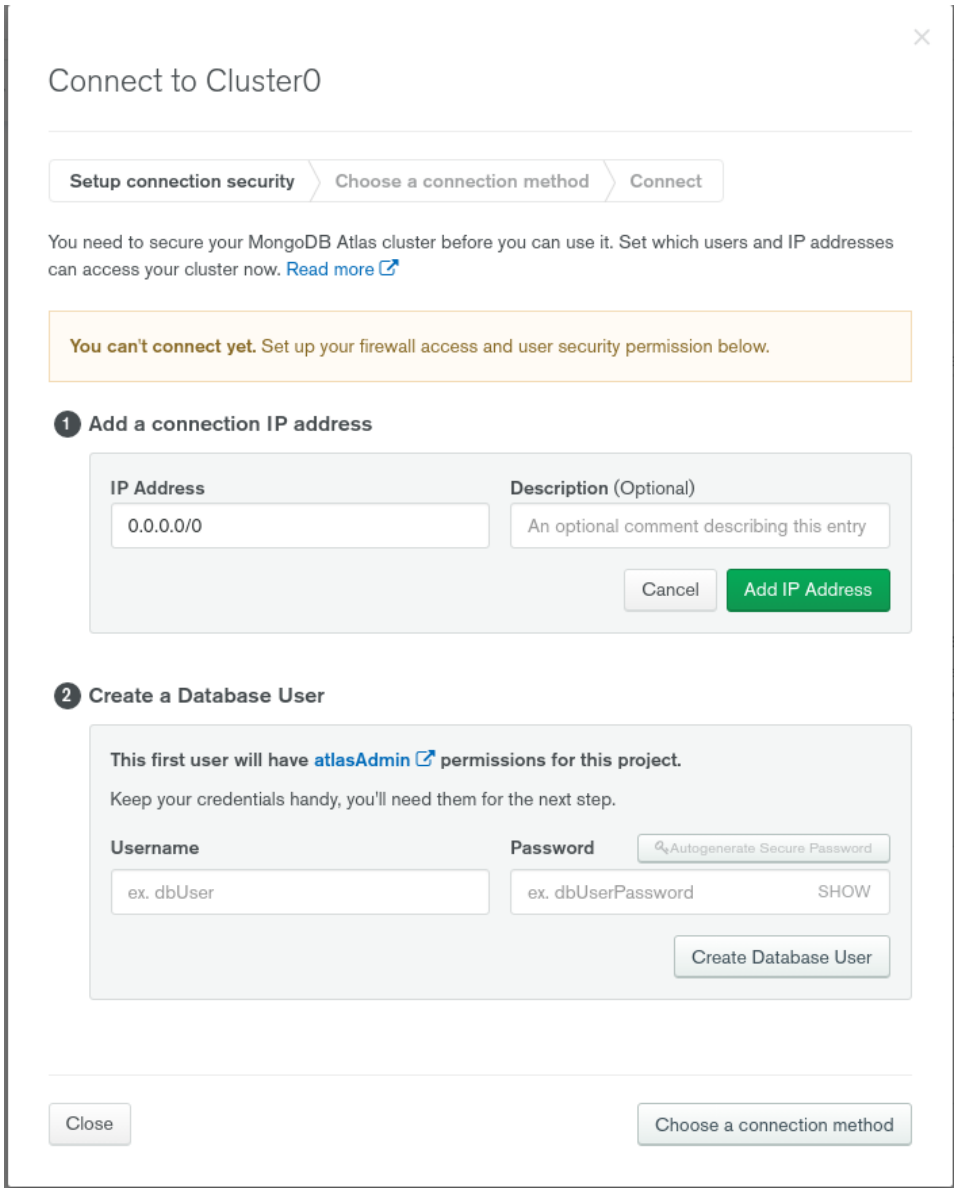


Figura 8. Configurar la conexión al Cluster en MongoDB Atlas - Paso 2

ADVERTENCIA

Guarda en un lugar seguro el usuario y la contraseña que crees en este paso. La contraseña no se puede volver a consultar después y la necesitarás más adelante para construir la URI de conexión. No uses la contraseña de tu cuenta institucional.

11. A continuación, en el punto 2 **agrega un usuario y una contraseña**. Como usuario te sugiero que utilices tu usuario institucional. Presiona el botón **Create Database User** (Fig. 9)

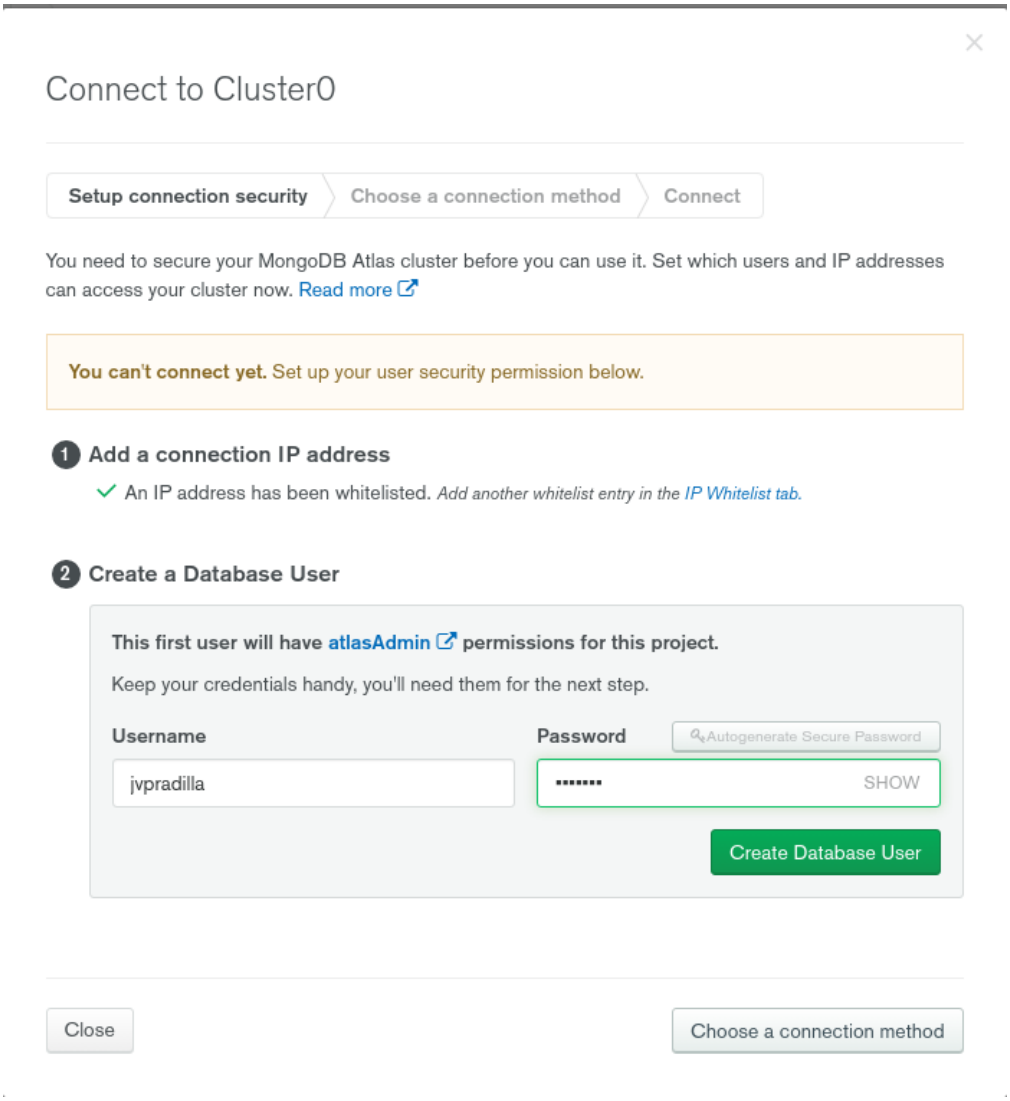


Figura 9. Configurar la conexión al Cluster en MongoDB Atlas - Paso 3

12. **Copia** el **usuario** y la **contraseña** que agregaste en el punto anterior:

```
Usuario:sebastiancossio111_db_user
```

```
Contraseña: Y6eRXaa2RMMAwZXk
```

13. Ahora **presiona** el botón **Choose a connection method** (Fig. 10)

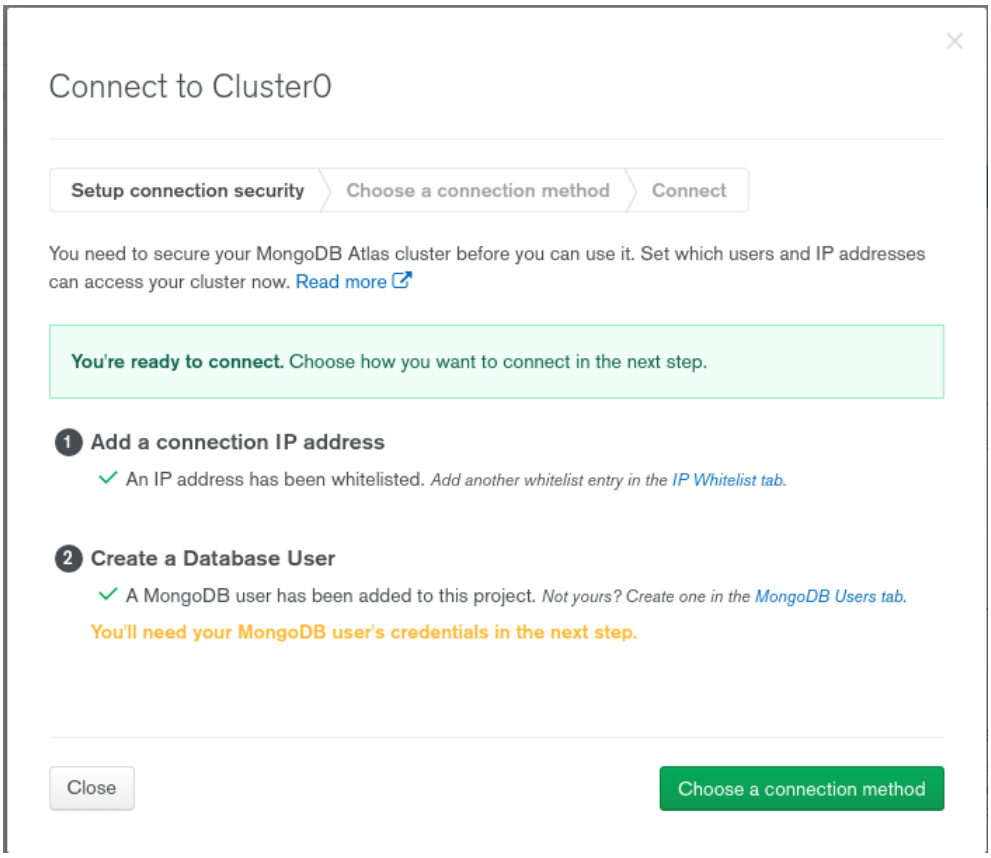


Figura 10. Configurar la conexión al Cluster en MongoDB Atlas - Paso 4

14. De las opciones presentadas **selecciona** Connect Your Application (Fig. 11)

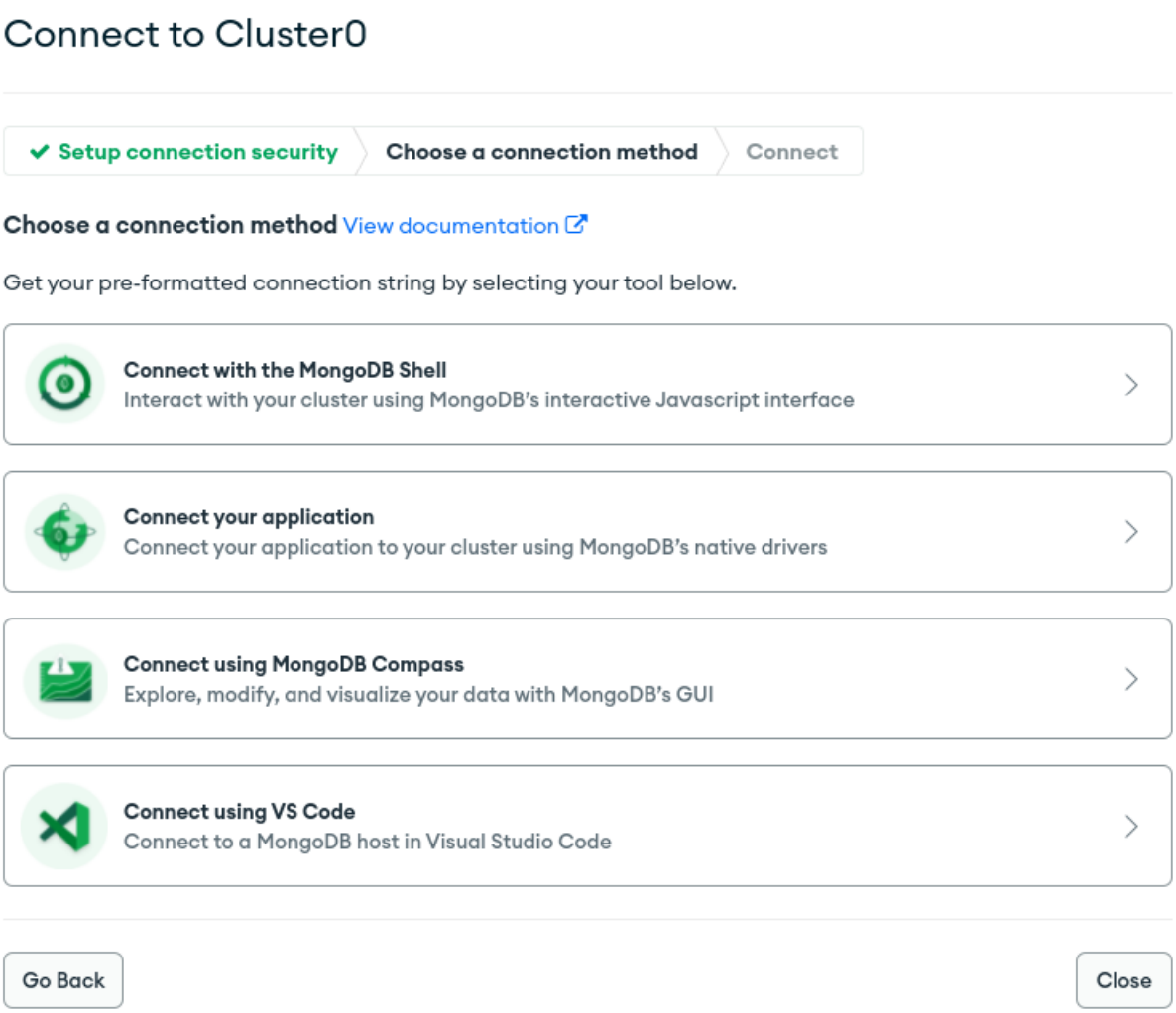


Figura 11. Configurar la conexión al Cluster en MongoDB Atlas - Paso 5

15. Finalmente, presiona el botón Close (Fig. 12)

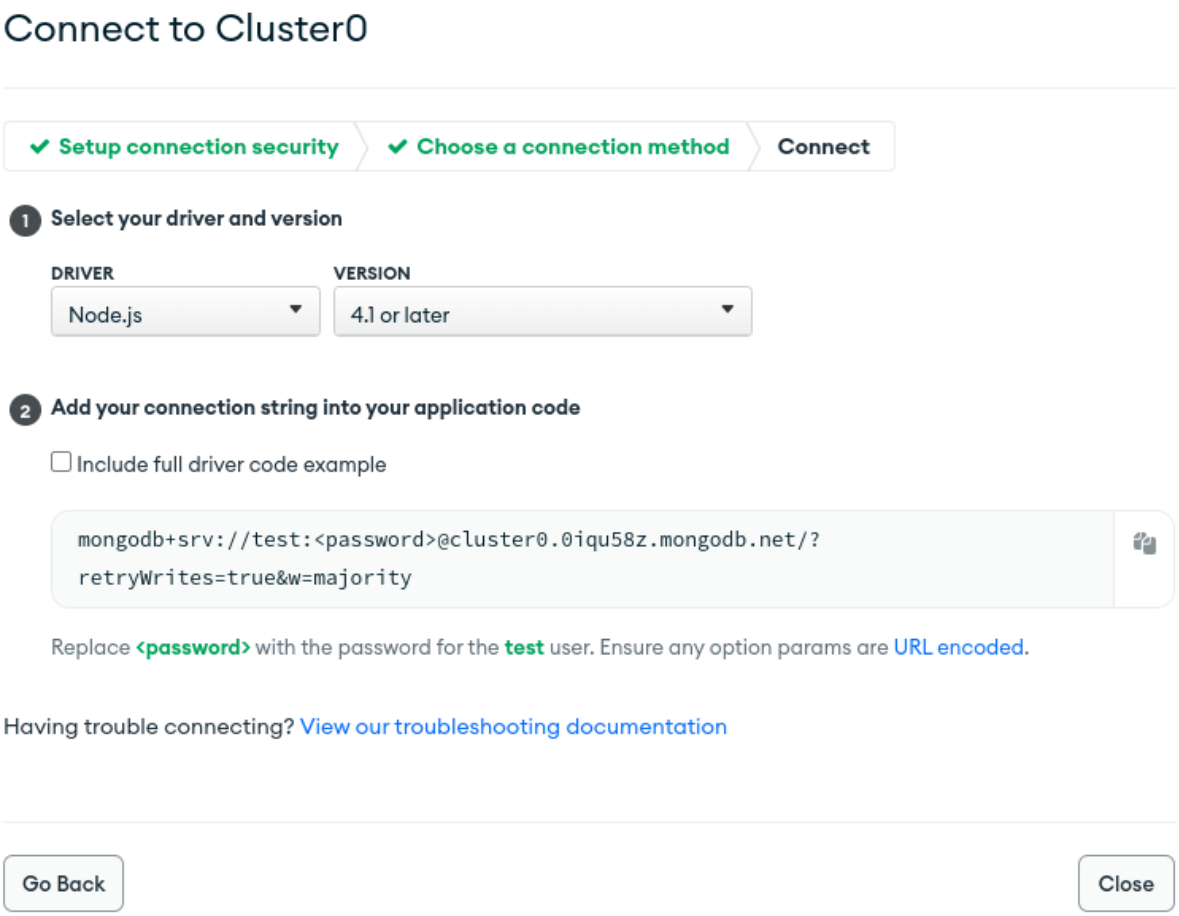


Figura 12. Configurar la conexión al Cluster en MongoDB Atlas - Paso 6

16. Copia la URI de conexión al Cluster:

```
URI: mongodb+srv://<db_username>:<db_password>@cluster0.izg3rja.mongodb.net/
```

Creando un proyecto en GitHub Codespaces

En esta práctica utilizaremos GitHub Codespaces, un entorno de desarrollo en la nube basado en Visual Studio Code, para trabajar con node.js, express.js y Javascript; de forma que se logre una conexión a la base de datos.

> Express.js
>
> Express.js es un framework para aplicaciones Web Node.js. Dicho ofrece dentro de sus posibilidades, el soporte necesario para crear API RESTful usando Javascript. Para profundizar en las capacidades y entender en profundidad el tutorial, debes estudiar la documentación oficial.

1. Desde el navegador Web **accede a la página** de GitHub (https://github.com/)
2. Inicia sesión en la aplicación, usando las credenciales de la cuenta de GitHub que creaste anteriormente (botón Sign in, en la parte superior derecha)
3. Accede a la página de GitHub Codespaces (https://github.com/codespaces) y presiona el botón New codespace
4. En la sección Quick start templates selecciona la plantilla Blank y presiona el botón Use this template
5. Espera unos minutos a que el Codespace sea creado y se abra el editor de código (Visual Studio Code en el navegador)

Creando una API básica con Express.js para conectar a la Base de Datos

1. Abre una terminal desde el menú Terminal / New Terminal e instala las dependencias del proyecto ejecutando el siguiente comando:

Terminal

```
npm install express@4.17.1 mongodb@3.6.2
```

2. Crea en la raíz del proyecto un archivo llamado index.js, usando el panel Explorer (botón New File)
3. Copia en el archivo index.js, usando el editor de código, el siguiente código:

> URI - CONTRASEÑA
>
> Debe reemplazar en el código el valor de la constante **uri** por la URI de la conexión al Cluster que copió anteriormente y completarla la **contraseña** que estableció para el usuario

index.js

```
// configuración del API RESTful
const express = require("express");
const app = express();
const port = 3000;

// configuración a la base de datos
const MongoClient = require("mongodb").MongoClient;
let ObjectId = require("mongodb").ObjectID;
const uri = "URI";

// Inicia la conexión a la base de datos
MongoClient.connect(uri, { useUnifiedTopology: true })
  .then((client) => {
    console.log("Conectado a la base de datos");
    const database = client.db("tutorial10");

    // Utiliza el parser para objetos JSON
    app.use(express.json());

    // Retorna todos los sensores almacenados
    app.get("/sensores", (req, res) => {
      database
        .collection("sensores")
        .find()
        .toArray()
        .then((sensors) => {
          res.status(200).send(sensors);
        })
        .catch((error) => console.error(error));
    });

    // Almacena un nuevo sensor
    app.post("/sensores", (req, res) => {
      let sensor = req.body;
      const sensorsCollection = database.collection("sensores");
      sensorsCollection
        .insertOne(sensor)
        .then((result) => {
          console.log(sensor);
          res.status(201).send(sensor);
        })
        .catch((error) => console.error(error));
    });

    // Retorna el sensor con el id pasado en el parámetro sensorId
    app.get("/sensores/:sensorId", (req, res) => {
      let sensorId = req.params.sensorId;
      database
        .collection("sensores")
        .findOne({ _id: new ObjectId(sensorId) })
```

```
      .then((sensor) => {
        if (null !== sensor) {
          res.status(200).send(sensor);
        } else {
          res.status(404).send(`El sensor con id: ${sensorId} no existe`);
        }
      })
      .catch((error) => {
        console.log(error);
      });
  });

  // Almacena una nueva medición
  app.post("/sensores/:sensorId/mediciones", (req, res) => {
    let sensorId = req.params.sensorId;
    let measurement = req.body;

    const updateSensor = {
      $addToSet: { mediciones: measurement }
    };

    database
      .collection("sensores")
      .updateOne({ _id: new ObjectId(sensorId) }, updateSensor)
      .then((sensor) => {
        if (null !== sensor) {
          console.log(sensor);
          res.status(201).send(measurement);
        } else {
          res.status(404).send(`El sensor con id: ${sensorId} no existe`);
        }
      })
      .catch((error) => {
        console.log(error);
      });
  });

  // Inicializa el servidor para que escuche en el puerto determinado
  app.listen(port, () => {
    console.log(`Servidor escuchando en el puerto: ${port}`);
  });
})
.catch((error) => console.error("Error al conectar a la base de datos"));
```

4. Guarda el archivo mediante el menú File/Save o usando el atajo de teclado Ctrl+s
5. Ejecuta la API escribiendo en la terminal el comando node index.js y observa su salida (Fig. 13).

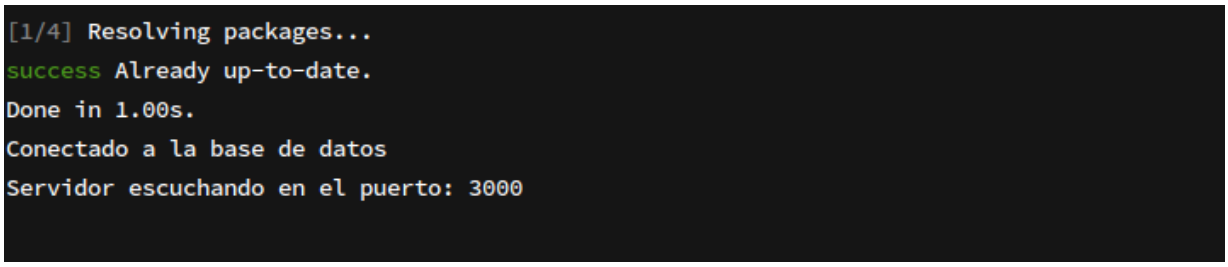


Figura 13. Salida de la Terminal del proyecto

6. Asegurate de que obtienes el mensaje: Conectado a la base de datos
7. Asegurate de que obtienes el mensaje: Servidor escuchando en el puerto: 3000
8. En el panel inferior abre la pestaña PORTS, haz clic derecho sobre el puerto 3000 y, en Port Visibility, selecciona Public (así podrás consumir la API desde Boomerang - SOAP & REST Client)
9. Copia la dirección que aparece en la columna Forwarded Address del puerto 3000 (URL pública de la API):

```
URL de la API RESTful: https://spindly-shadow-v6prv65ww7vg3wpjj-
3000.app.github.dev/sensores
```

Probando la API RESTFul

1. Instala la aplicación Boomerang - SOAP & REST Client desde la tienda oficial de Chrome (Fig. 14)

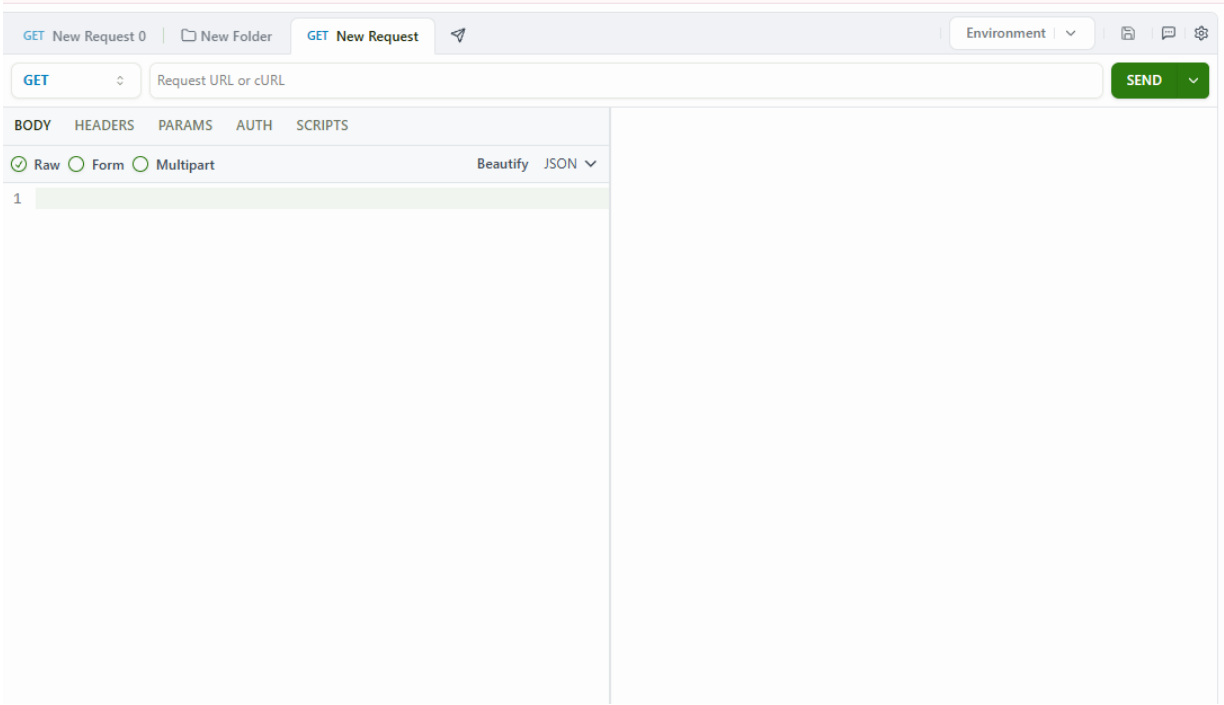


Figura 14. Boomerang - SOAP & REST Client

2. **Copia** la **URL** de la API RESTful que obtuviste desde GitHub Codespaces en la barra de dirección (junto al método) y agregale **/sensores/** al final de la misma (Fig. 15)
3. Cambia el método por **POST** (Fig. 15)
4. En la pestaña **Body**, selecciona **Raw y el formato JSON** (Fig. 15)
5. **Copia** la siguiente información en el campo de Body

body

```
{
 "nombre":"Sensor de Temperatura - UAO - InfoLab"
}
```

6. **Presiona** el botón Send
7. En la pestaña **preview** debes obtener una salida como la siguiente: (Fig. 15)

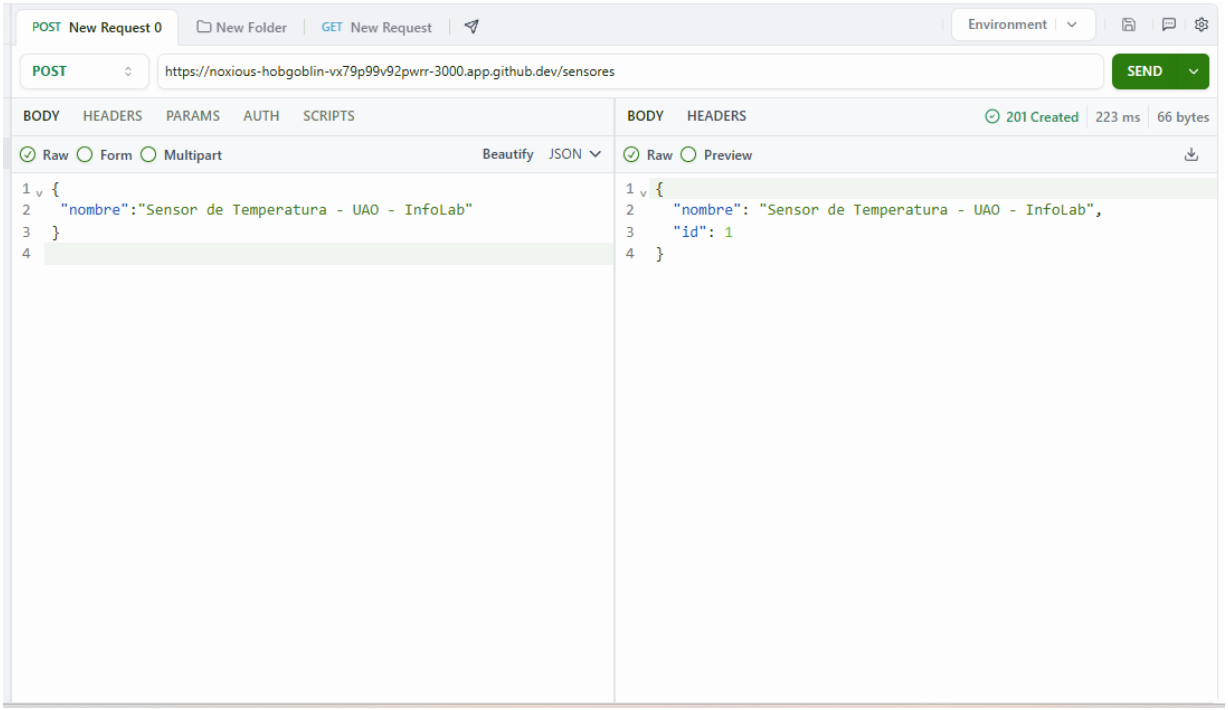


Figura 15. Resultado de la operación

8. **Copia** el **_id** asignado por la base de datos al sensor

```
_id: 6ae20138701b65f5814cfeda
```

9. En el navegador **ingresa** la página Web de [MongoDB Atlas]
10. **Selecciona** la opción **Clusters** en el menú de la izquierda (Fig. 16)
11. **Selecciona COLLECTIONS** en el Cluster que creaste para el proyecto (Fig. 16)

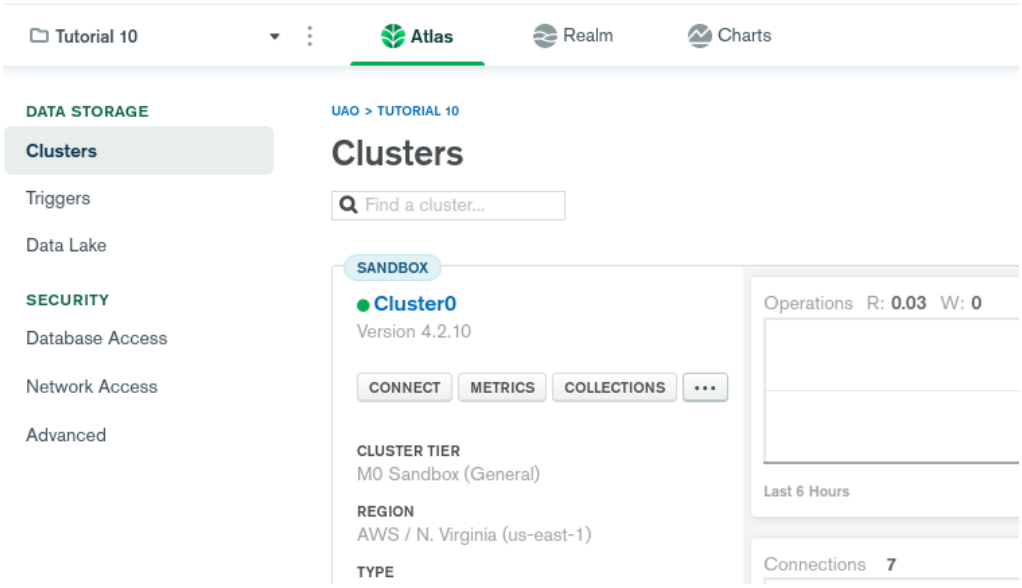


Figura 16. Ventana de Clusters en MongoDB Atlas

12. En la ventana que se desplegó podrás **ver** la **colección sensores** y el **Documento sensor** en formato **JSON** que acabas de ingresar con el _id dado por la base de datos (Fig. 17)

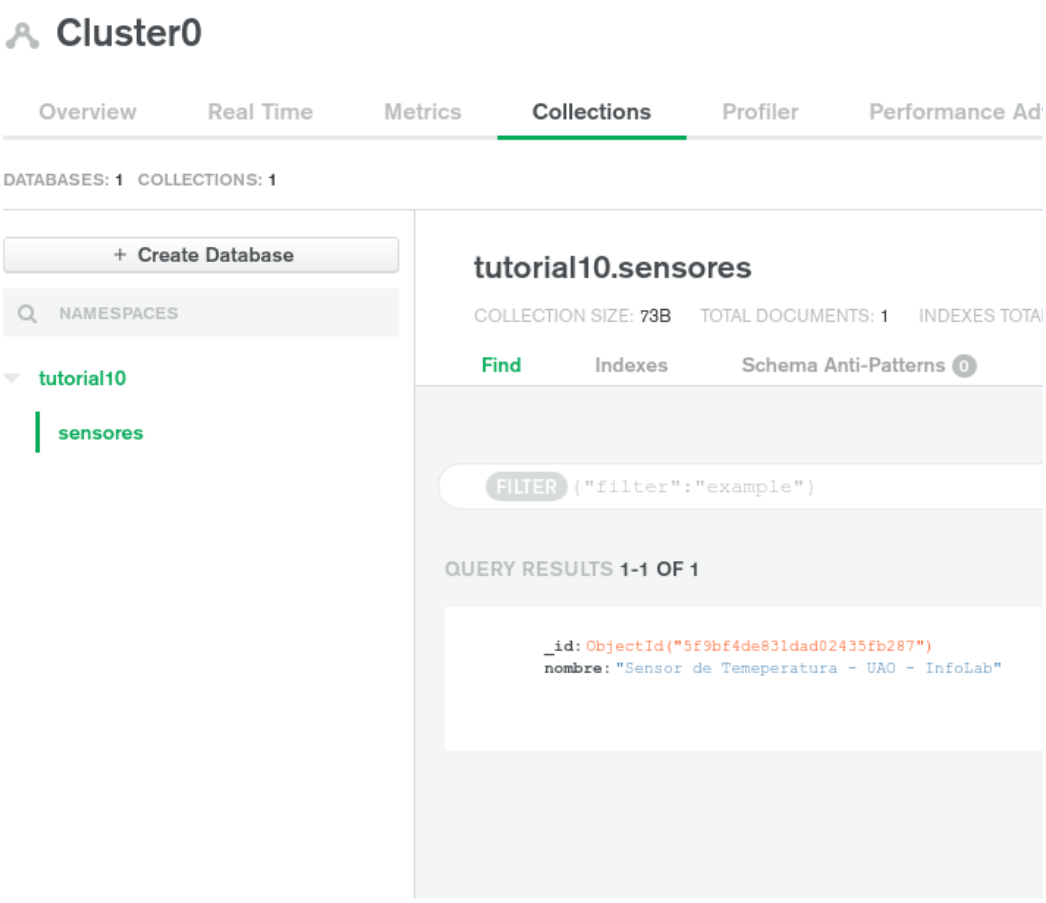


Figura 17. Colección sensores en MongoDB Atlas

13. **Copia** la **URL** de la API RESTful que obtuviste desde GitHub Codespaces en la barra de dirección (junto al método) y agregale **/sensores** al final de la misma (Fig. 18)
14. Cambia el método por **GET** (Fig. 18)
15. **Presiona** el botón Send
16. En la pestaña **preview** debes obtener una salida como la siguiente: (Fig. 18).

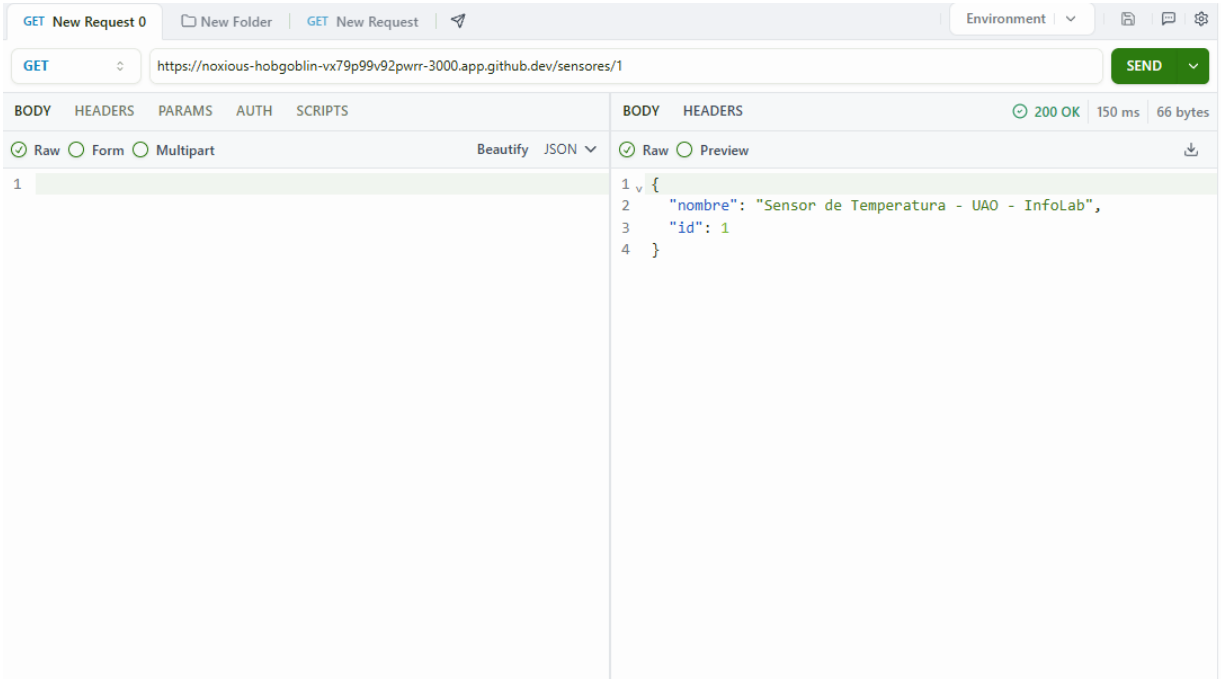


Figura 18. Resultado de la operación

17. **Copia** la **URL** de la API RESTful que obtuviste desde GitHub Codespaces en la barra de dirección (junto al método) y agregale **/sensores/<_id>/mediciones** al final de la misma. Donde <_id> es el id asignado al sensor por la base de datos (Fig. 19)
18. Cambia el método por **POST** (Fig. 19)
19. En la pestaña **Body**, selecciona **Raw y el formato JSON** (Fig. 19)
20. **Copia** la siguiente información en el campo de Body

body

```
{
   "momento": "2020-10-23T20:51:39.720Z",
   "valor": 27.8,
   "unidad": "ºC"
}
```

21. **Presiona** el botón Send
22. En la pestaña **preview** debes obtener una salida como la siguiente: (Fig. 19)

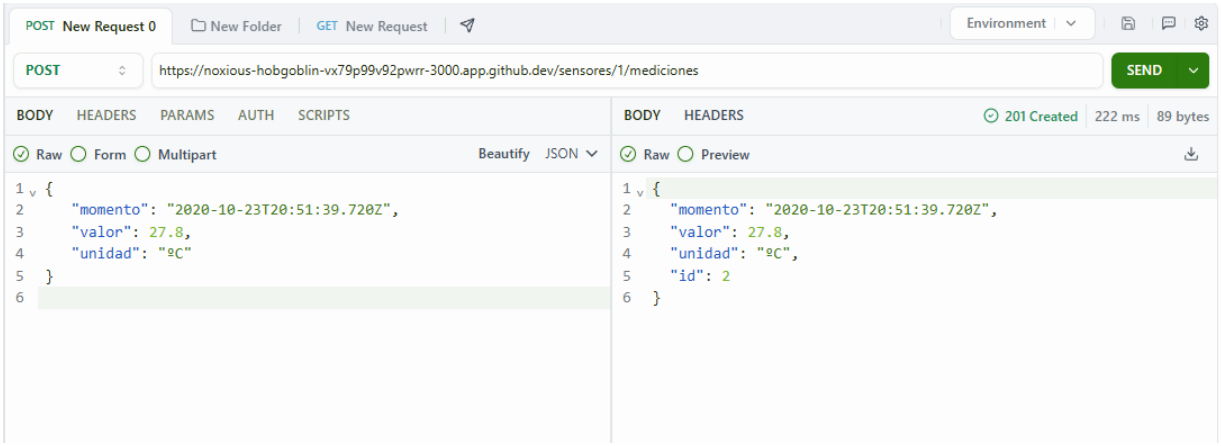


Figura 19. Resultado de la operación

23. Ve a la página Web de **MongoDB Atlas** y refresca las colecciones deberías ver el **Documento sensor** con la **medición** que acabas de ingresar (Fig. 20)

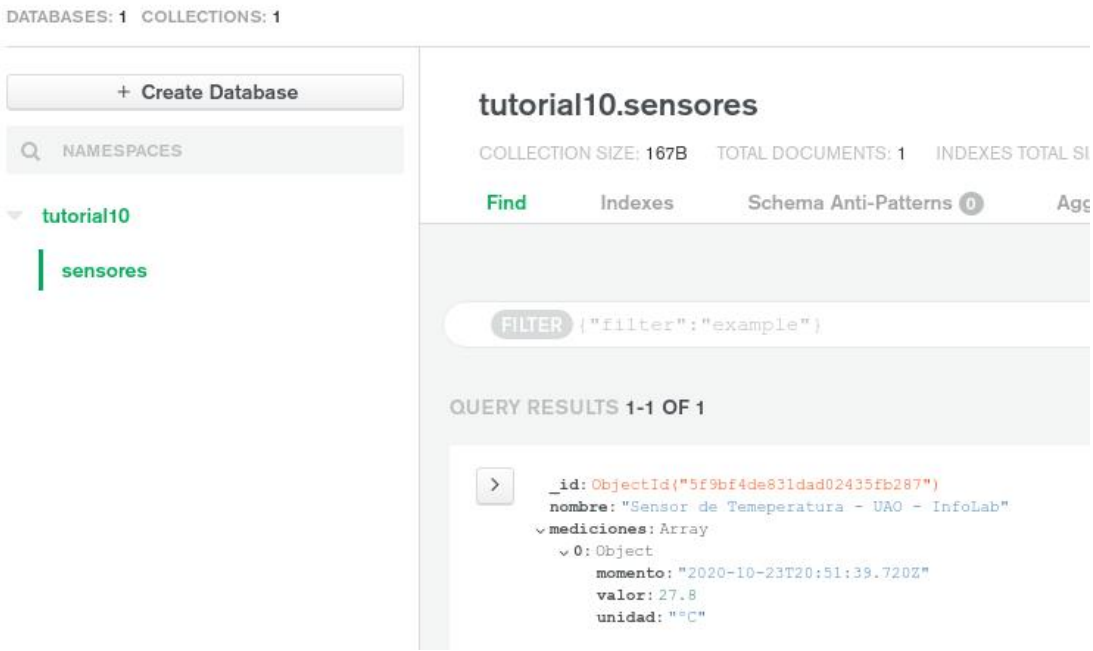
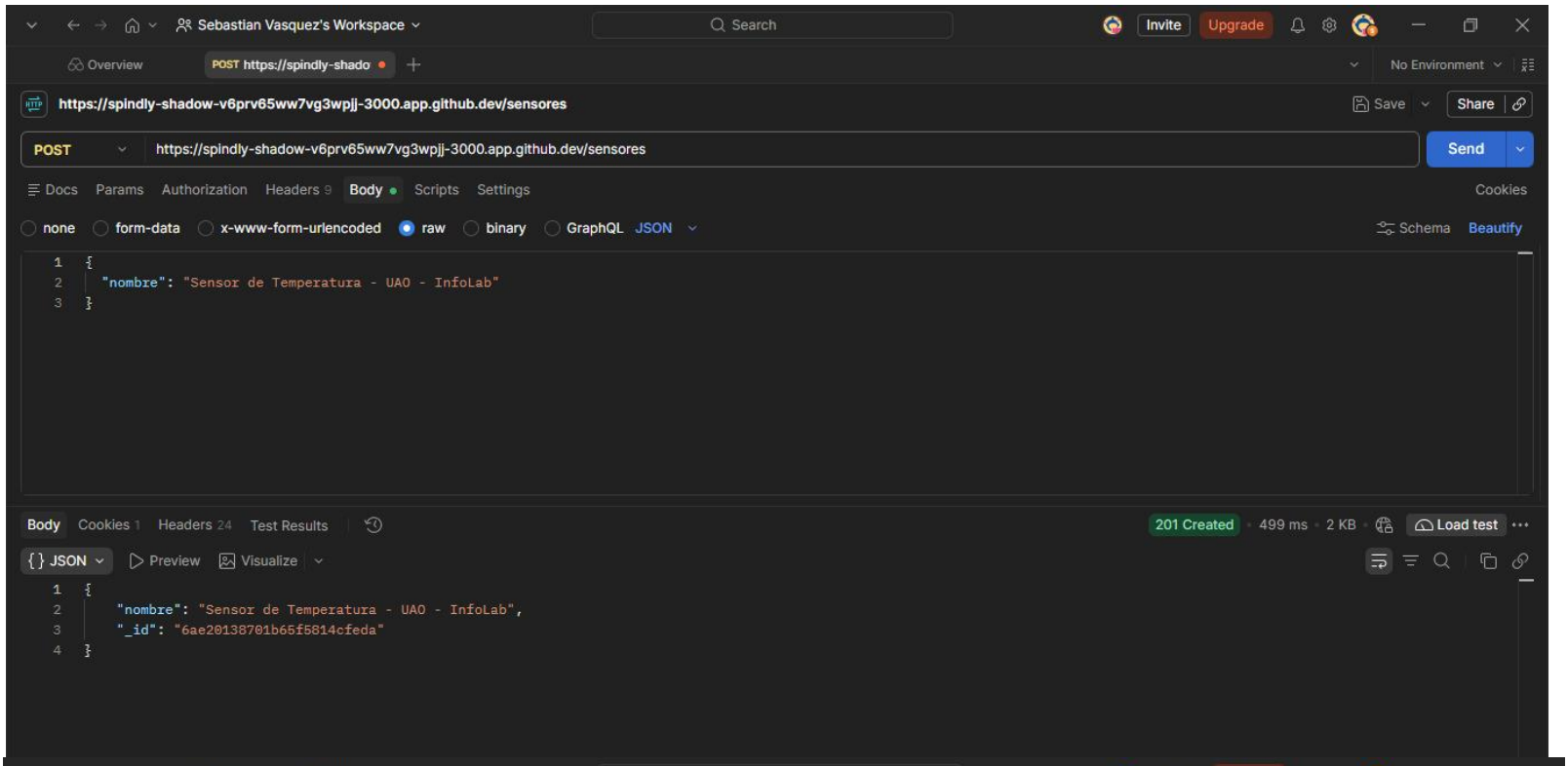
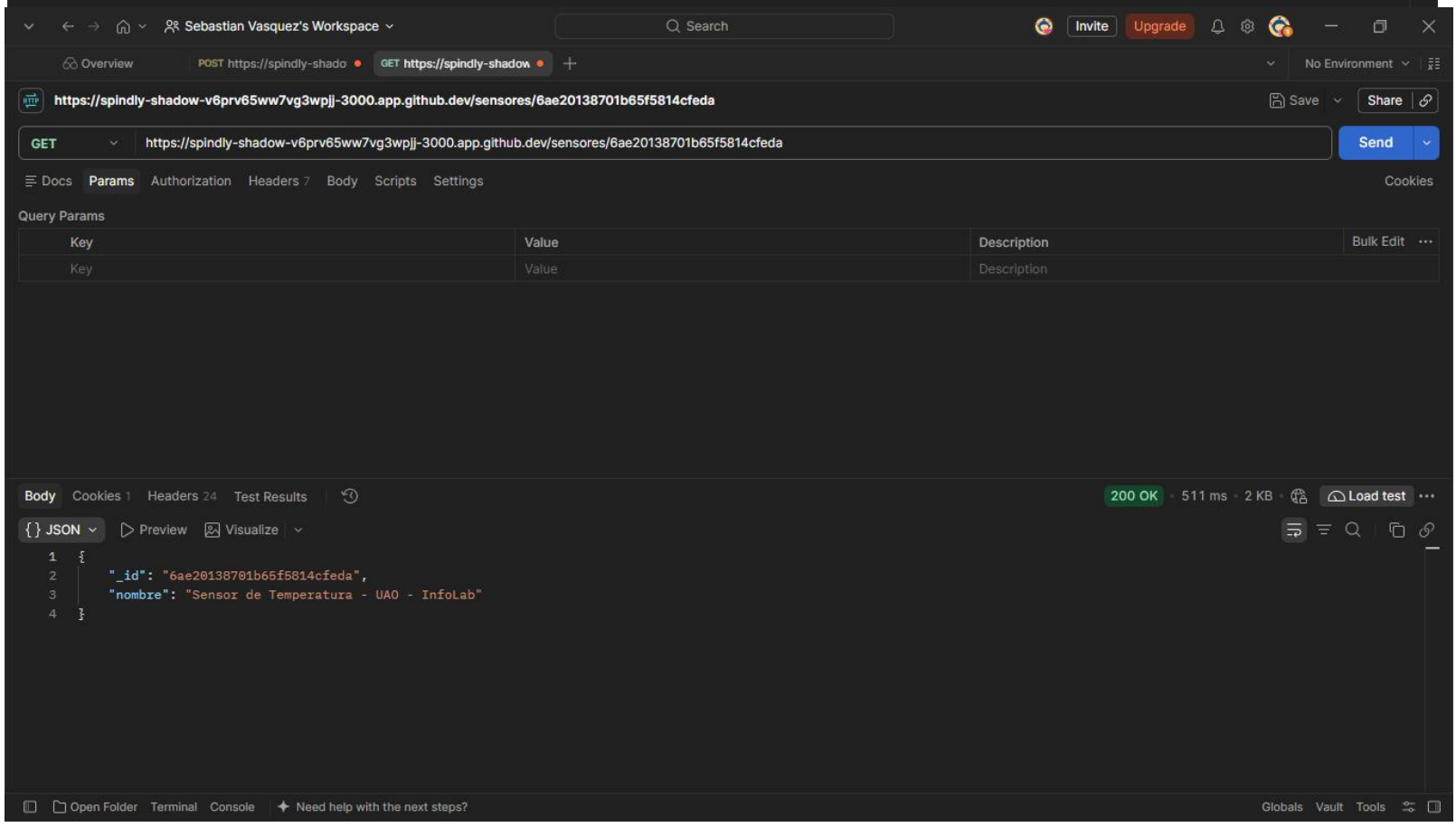
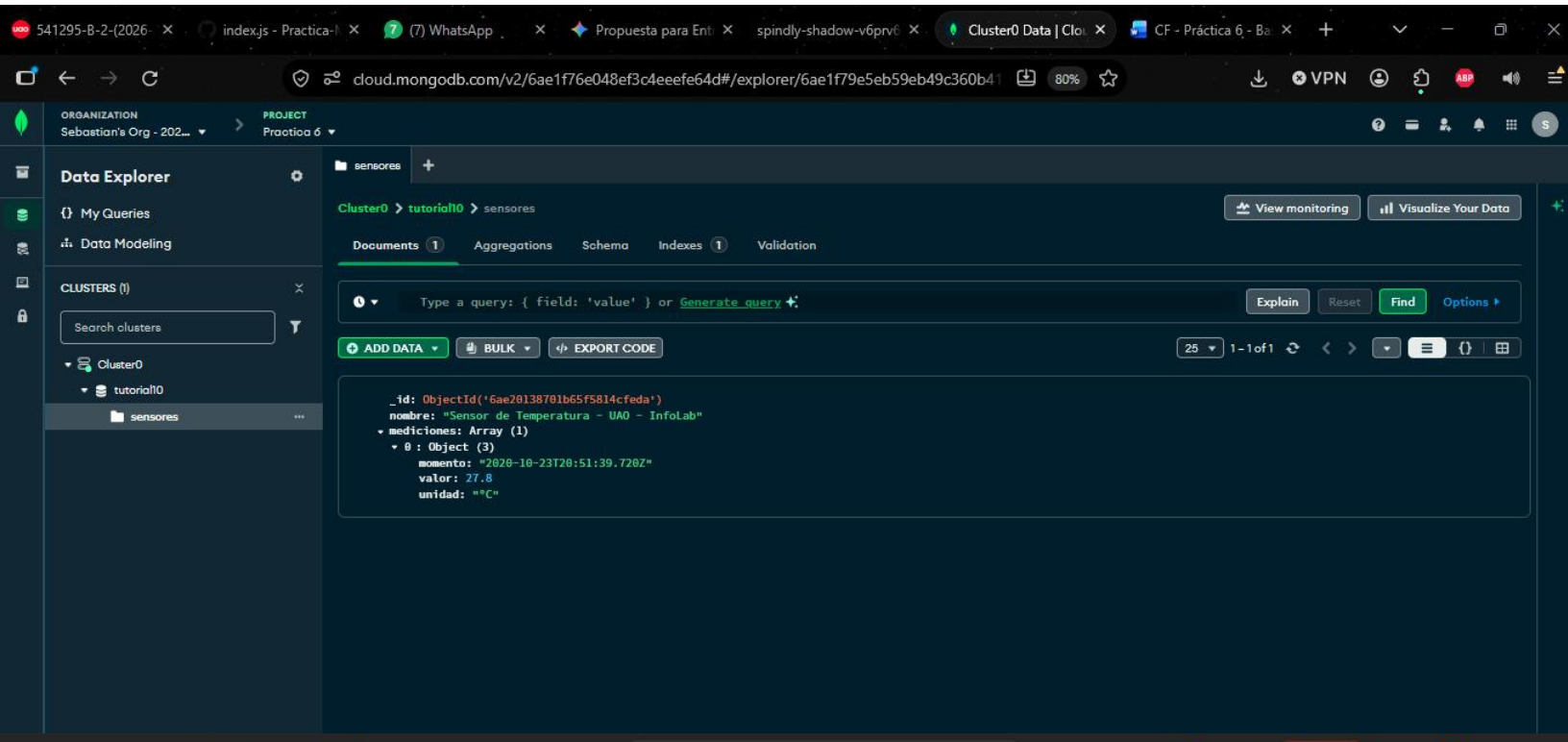


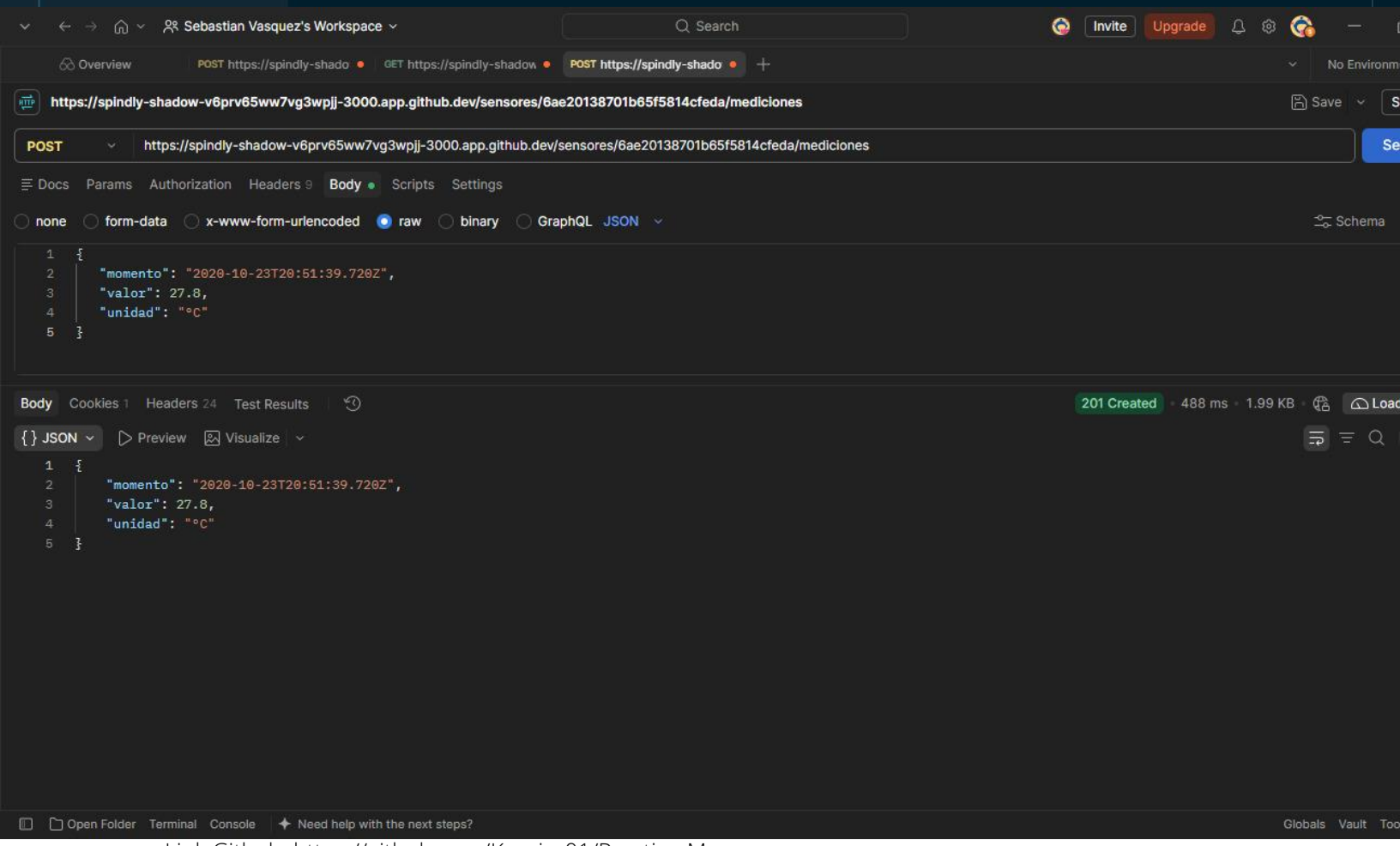
Figura 20. Colección sensores en MongoDB Atlas

Pruebas


Sebastian Vasquez's Workspace
Search
Invite
Upgrade
Overview
POST https://spindly-shado
No Environment
https://spindly-shadow-v6prv65ww7vg3wpjj-3000.app.github.dev/sensores
Save
Share
POST
https://spindly-shadow-v6prv65ww7vg3wpjj-3000.app.github.dev/sensores
Send
Docs Params Authorization Headers 9 Body Scripts Settings
Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
{
"nombre": "Sensor de Temperatura - UAO - InfoLab"
}
Body Cookies 1 Headers 24 Test Results
201 Created 499 ms 2 KB Load test
{} JSON Preview Visualize
{
"nombre": "Sensor de Temperatura - UAO - InfoLab",
"_id": "6ae20138701b65f5814cfeda"
}


Sebastian Vasquez's Workspace
Search
Invite
Upgrade
Overview
POST https://spindly-shado
GET https://spindly-shadow
No Environment
https://spindly-shadow-v6prv65ww7vg3wpjj-3000.app.github.dev/sensores/6ae20138701b65f5814cfeda
Save
Share
GET
https://spindly-shadow-v6prv65ww7vg3wpjj-3000.app.github.dev/sensores/6ae20138701b65f5814cfeda
Send
Docs Params Authorization Headers 7 Body Scripts Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
Key
Value
Description
Body Cookies 1 Headers 24 Test Results
200 OK 511 ms 2 KB Load test
{} JSON Preview Visualize
{
"_id": "6ae20138701b65f5814cfeda",
"nombre": "Sensor de Temperatura - UAO - InfoLab"
}
Open Folder Terminal Console
Need help with the next steps?
Globals Vault Tools





Link Github: https://github.com/Kossio-01/Practica-Mongo